

Package ‘repMeta’

September 21, 2026

Type Package

Title {repMeta}: Quantifying Replicability of Multiple Studies in a Meta-Analysis

Version 0.1.0

Depends R (>= 3.5),

Imports parallel (>= 3.5.3),
evd (>= 2.3.3),
metafor (>= 3.8.1),
mathjaxr

RdMacros mathjaxr

Author Mengli Xiao, Haitao Chu, James S. Hodges, and Lifeng Lin

Maintainer Mengli Xiao <mengli.xiao@cuanschutz.edu>

Description The package provides functionality for replicability analysis of multiple studies in a meta-analysis, as described in detail in the paper Quantifying Replicability of Multiple Studies in a Meta-Analysis

License GPL (>= 2)

Encoding UTF-8

LazyData true

RoxygenNote 7.1.2

URL <https://github.com/menglix/repMeta>

BugReports <https://github.com/menglix/repMeta/issues>

Suggests testthat (>= 3.0.0)

Config/testthat/edition 3

R topics documented:

calempF	2
calR	2
lahart18	3
moller12	4
pRmasym	4
pRmboot	5
Rm.func.iterative	5
Rm.func.iterative.boot	6
to.dat.repMeta	7

Index	10
--------------	-----------

calempF	<i>Obtain multiple R_m values from parametric bootstrap</i>
---------	--

Description

This function obtains numbers of nb R_m values from parametric bootstrap

Usage

```
calempF(dat1, m, nb = 1000, n.cores = getOption("mc.cores", 2L))
```

Arguments

dat1	data frame, a meta-analysis dataset with after to.dat.repMeta .
m	numeric, m value.
nb	numeric, number of iterations in parametric bootstrap.

Value

R_m values from parametric bootstrap, a vector of length nb .

Examples

```
data.case <- to.dat.repMeta(data=moller12,ai = r1, n1i = n1, ci = r2, n2i = n2,measure="OR")
# 1000 bootstrapped  $\{R_1\}$  values
calempF(data.case,1)
# 500 bootstrapped  $\{R_1\}$  values
calempF(data.case,1,nb=500)
```

calR	<i>Quantify replicability in a meta-analysis by the observed R_m statistic</i>
------	---

Description

This function obtains the observed R_m statistic to quantify replicability of all studies in a meta-analysis. The meta-analysis dataset needs be transformed by [to.dat.repMeta](#) before using this function.

Usage

```
calR(dat1, m = 1)
```

Arguments

dat1	data.frame, a meta-analysis dataset with n studies after transformed by to.dat.repMeta .
m	numeric, m value.

Value

A list contains the observed R_m and C_m^n values of $R_{\mathcal{A}_{m,k}}$ from the meta-analysis.

`maxR` the observed R_m .

`Rvals` $R_{\mathcal{A}_{m,k}}$ values, a vector of length C_m^n .

Examples

```
# Obtain the R1
data.case <- to.dat.repMeta(data=moller12,ai = r1, n1i = n1, ci = r2, n2i = n2,measure="OR")
calR(data.case,1)
```

lahart18	<i>Individual format physical activity versus control for reducing body fat in women with breast cancer after adjuvant therapy</i>
----------	--

Description

A meta-analysis dataset compared individual format physical activity versus control for reducing body fat in women with breast cancer after adjuvant therapy in 10 studies, and the effect measure is the mean difference.

Usage

```
lahart18
```

Format

A data frame with 10 rows and 7 variables:

`CDSR.id` character, CDSR ID

`data.type` character, show whether the outcome is continous (CONT) or binary (DICH)

`study.name` character, author and year

`y` numeric, effect size

`s2` numeric, variance of the effect size

`n1` numeric, number of subjects in the treatment group

`n2` numeric, number of subjects in the control group

Source

<https://www.cochranelibrary.com/cdsr/doi/10.1002/14651858.CD011292.pub2/full>

moller12	<i>off-pump with heparin dose less than 300 mg/k vs. on-pump coronary artery bypass grafting (CABG) for reducing postoperative atrial fibrillation (POAF)</i>
----------	---

Description

A meta-analysis dataset compared off-pump with heparin dose less than 300 mg/k vs. on-pump coronary artery bypass grafting (CABG) for reducing postoperative atrial fibrillation (POAF) in 17 studies, the effect measure we used is the odds ratio (OR).

Usage

```
moller12
```

Format

A data frame with 17 rows and 7 variables:

CDSR.id character, CDSR ID

data.type character, show whether the outcome is continous (CONT) or binary (DICH)

study.name character, author and year

r1 numeric, event counts in the treatment group

r2 numeric, event counts in the control group

n1 numeric, number of subjects in the treatment group

n2 numeric, number of subjects in the control group

Source

<https://www.cochranelibrary.com/cdsr/doi/10.1002/14651858.CD007224.pub2/full>

pRmasym	<i>Test the replicability of multiple studies in a meta-analysis from R_m (the Gumbel approximation)</i>
---------	---

Description

This function obtains the p -value of the replicability test (detect non-replicability), using the Gumbel approximation. The meta-analysis dataset needs be transformed by [to.dat.repMeta](#).

Usage

```
pRmasym(dat1, m)
```

Arguments

dat1 data.frame, a meta-analysis dataset with n studies after transformed by [to.dat.repMeta](#).
 m numeric, m value.

Value

The p -value of R_m from the Gumbel approximation.

Examples

```
# Obtain the R1
data.case <- to.dat.repMeta(data=moller12,ai = r1, n1i = n1, ci = r2, n2i = n2,measure="OR")
pRmasym(data.case,1)
```

pRmboot	<i>Test the replicability of multiple studies in a meta-analysis from R_m (the parametric bootstrap)</i>
---------	---

Description

This function obtains the p -value of the replicability test (detect non-replicability), using the parametric bootstrap. The meta-analysis dataset needs be transformed by [to.dat.repMeta](#).

Usage

```
pRmboot(dat1, m = 1)
```

Arguments

dat1	data.frame, a meta-analysis dataset with n studies after transformed by to.dat.repMeta .
m	numeric, m value.

Value

The p -value of R_m from the parametric bootstrap.

Examples

```
# Obtain the R1
data.case <- to.dat.repMeta(data=moller12,ai = r1, n1i = n1, ci = r2, n2i = n2,measure="OR")
pRmboot(data.case,1)
```

Rm.func.iterative	<i>Identify non-replicable studies in a meta-analysis from R_m (the Gumbel approximation)</i>
-------------------	--

Description

This function identifies non-replicable studies from R_m and replicability test (using the Gumbel approximation). The meta-analysis dataset needs be transformed by [to.dat.repMeta](#).

Usage

```
Rm.func.iterative(dat1, m = 1)
```

Arguments

`dat1` data.frame, a meta-analysis dataset with n studies after transformed by [to.dat.repMeta](#).
`m` numeric, m value, the default is 1.

Value

A list object containing following components:

`stat` a vector of the R_m value and its corresponding p -value after non-replicable studies are removed.

`P_1` p -value of replicability test for the dataset using the Gumbel approximation.

`nonrep_studies` Row indices of the non-replicable studies in `dat1`

Examples

```
# Identify the non-replicable study using \eqn{R_1}
data.case <- to.dat.repMeta(data=moller12,ai = r1, n1i = n1, ci = r2, n2i = n2,measure="OR")
iden.ls <- Rm.func.iterative(data.case,1)
# \eqn{p}-value of replicability test
iden.ls$P_1
# index of non-replicable study
iden.ls$nonrep_studies
nonrep.id <- iden.ls$nonrep_studies
# \eqn{R_1} and its \eqn{p}-value among replicable studies
iden.ls$stat
```

Rm.func.iterative.boot

Identify non-replicable studies in a meta-analysis from R_m (the parametric bootstrap)

Description

This function identifies non-replicable studies from R_m and replicability test (using the parametric bootstrap). The meta-analysis dataset needs be transformed by [to.dat.repMeta](#).

Usage

```
Rm.func.iterative.boot(dat1, m = 1)
```

Arguments

`dat1` data.frame, a meta-analysis dataset with n studies after transformed by [to.dat.repMeta](#).
`m` numeric, m value, the default is 1.

Value

A list object containing following components:

`stat` a vector of the R_m value and its corresponding p -value after non-replicable studies are removed, using parametric bootstrap.

`P_1` p -value of replicability test for the dataset using parametric bootstrap.

`nonrep_studies` Row indices of the non-replicable studies in `dat1`

Examples

```
# Identify the non-replicable study using  $R_1$ 
data.case <- to.dat.repMeta(data=moller12,ai = r1, n1i = n1, ci = r2, n2i = n2,measure="OR")
iden.ls.boot <- Rm.func.iterative.boot(data.case,1)
#  $p$ -value of replicability test
iden.ls.boot$P_1
# index of non-replicable study
iden.ls.boot$nonrep_studies
nonrep.id <- iden.ls.boot$nonrep_studies
#  $R_1$  and its  $p$ -value among replicable studies
iden.ls.boot$stat
```

to.dat.repMeta	<i>Transform a meta-analysis dataset for replicability analysis.</i>
----------------	--

Description

This function transforms a meta-analysis dataset to the dataset compatible of the package. More details are in [escalc](#).

Usage

```
to.dat.repMeta(
  measure,
  ai,
  bi,
  ci,
  di,
  n1i,
  n2i,
  x1i,
  x2i,
  t1i,
  t2i,
  m1i,
  m2i,
  sd1i,
  sd2i,
  xi,
  mi,
  ri,
  ti,
  sdi,
  r2i,
  ni,
  yi,
  vi,
  sei,
  data = NULL,
  slab,
  subset,
```

```

include,
add = 1/2,
to = "only0",
drop00 = FALSE,
vtype = "LS",
var.names = c("yi", "vi"),
add.measure = FALSE,
append = TRUE,
replace = TRUE,
digits,
...
)

```

Arguments

measure	a character string to specify which effect size or outcome measure should be calculated. See Details in escalc for possible options and how the data needed to compute the selected effect size or outcome measure should then be specified (i.e., which of the following arguments need to be used).
ai	vector with the 2×2 table frequencies (upper left cell).
bi	vector with the 2×2 table frequencies (upper right cell).
ci	vector with the 2×2 table frequencies (lower left cell).
di	vector with the 2×2 table frequencies (lower right cell).
n1i	vector with the group sizes or row totals (first group/row).
n2i	vector with the group sizes or row totals (second group/row).
x1i	vector with the number of events (first group).
x2i	vector with the number of events (second group).
t1i	vector with the total person-times (first group).
t2i	vector with the total person-times (second group).
m1i	vector with the means (first group or time point).
m2i	vector with the means (second group or time point).
sd1i	vector with the standard deviations (first group or time point).
sd2i	vector with the standard deviations (second group or time point).
xi	vector with the frequencies of the event of interest.
mi	vector with the frequencies of the complement of the event of interest or the group means.
ri	vector with the raw correlation coefficients.
ti	vector with the total person-times or t-test statistics.
sdi	vector with the standard deviations.
r2i	vector with the R^2 values.
ni	vector with the sample/group sizes.
yi	vector with the observed effect sizes or outcomes.
vi	vector with the corresponding sampling variances.
sei	vector with the corresponding standard errors.
data	data frame containing the variables given to the arguments above.

slab	optional vector with labels for the studies.
subset	optional (logical or numeric) vector to specify the subset of studies that will be included in the data frame returned by the function.
include	optional (logical or numeric) vector to specify the subset of studies for which the measure should be calculated.
add	a non-negative number to specify the amount to add to zero cells, counts, or frequencies. See Details in escalc .
to	a character string to specify when the values under add should be added (either "all", "only0", "if0all", or "none"). See Details in escalc .
drop00	logical to specify whether studies with no cases/events (or only cases) in both groups should be dropped when calculating the observed effect sizes or outcomes. See Details in escalc .
vtype	a character string to specify the type of sampling variances to calculate. See Details in escalc .
var.names	character vector with two elements to specify the name of the variable for the observed effect sizes or outcomes and the name of the variable for the corresponding sampling variances (the defaults are "yi" and "vi").
add.measure	logical to specify whether a variable should be added to the data frame (with default name "measure") that indicates the type of outcome measure computed. When using this option, var.names can have a third element to change this variable name.
append	logical to specify whether the data frame provided via the data argument should be returned together with the observed effect sizes or outcomes and corresponding sampling variances (the default is TRUE).
replace	logical to specify whether existing values for yi and vi in the data frame should be replaced. Only relevant when append=TRUE and the data frame already contains the yi and vi variables. If replace=TRUE (the default), all of the existing values will be overwritten. If replace=FALSE, only NA values will be replaced.
digits	optional integer to specify the number of decimal places to which the printed results should be rounded. If unspecified, the default is 4. Note that the values are stored without rounding in the returned object.
...	other arguments in escalc .

Value

An object of class `c("escalc","data.frame")`.

Examples

```
# Obtain the transformed data frame format with odds ratio as the summary measure
data.case <- to.dat.repMeta(data=moller12,ai = r1, n1i = n1, ci = r2, n2i = n2,measure="OR")
# Similar to \code{metafor}, we can compute any other measure, such as logit transformed proportions under exam
to.dat.repMeta(measure="PLO", xi=c(5,12), ni=c(40,80))
```

Index

*Topic **datasets**

lahart18, [3](#)

moller12, [4](#)

calempF, [2](#)

calR, [2](#)

escalc, [7–9](#)

lahart18, [3](#)

moller12, [4](#)

pRmasym, [4](#)

pRmboot, [5](#)

Rm.func.iterative, [5](#)

Rm.func.iterative.boot, [6](#)

to.dat.repMeta, [2](#), [4–6](#), [7](#)